DOCUMENTAÇÃO TÉCNICA

API de Integração Infominas Desk

Como conectar um chatbot ou IA de atendimento ao Desk: webhooks de mensagens, WhatsApp, triagem, chamados e passagem para atendentes.

Versão da API: v1 · **Atualizado em:** 19 de outubro de 2026
https://infodesk.infominas.net/v1/integration/v1

Sumário

Visão geral

Passo a passo no Desk: criar a chave

2. Autenticação e escopos

3. Erros

4. Paginação

5. Eventos e webhooks

6. WhatsApp e conversas

7. Bot externo (triagem)

8. Contatos e solicitantes

9. Chamados

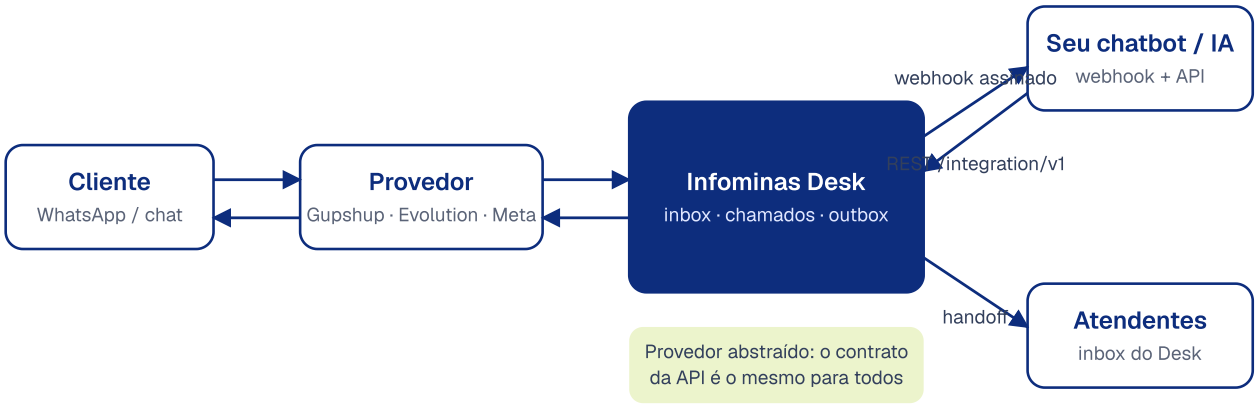
10. Fluxo de referência: bot de triagem

11. Outras rotas

12. Boas práticas

Visão geral

A API de Integração permite que um sistema externo, em especial um **chatbot ou IA de atendimento**, opere o Infominas Desk como um atendente: recebe cada mensagem que chega, responde no WhatsApp, identifica o cliente, abre e conduz chamados e passa o atendimento para a equipe quando precisa.



Saída: eventos

Toda mensagem, mudança de conversa e de chamado vira um evento entregue por `webhook` assinado, com reentrega automática e feed retroativo.

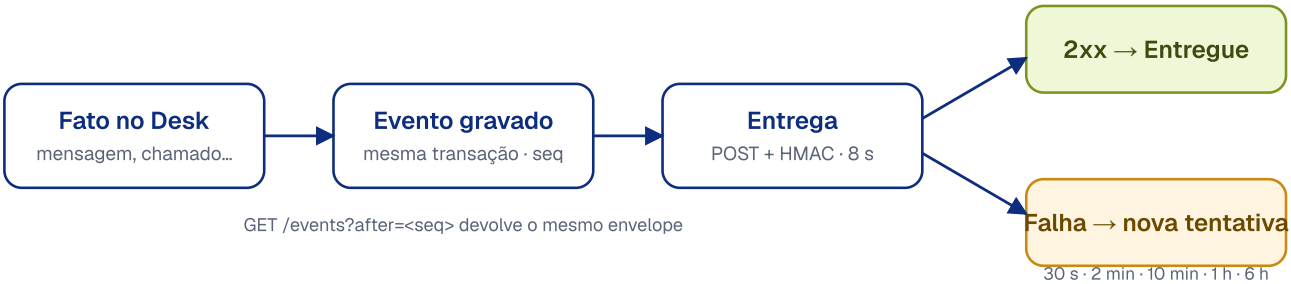
Entrada: ações

REST em `/integration/v1`: enviar WhatsApp (texto, mídia, template), operar conversas e chamados, buscar contatos.

Triagem: bot externo

O atendimento novo nasce com o seu bot, sem incomodar a equipe, até o `handoff`.

Como a entrega de eventos funciona



O fluxo de um atendimento com o bot



O **provedor de WhatsApp** (Gupshup, Evolution ou Meta) é invisível para a integração: templates são chamados pelo nome, o status de entrega é sempre `sent → delivered → read` (ou `failed`) e os erros vêm com códigos do Desk. Trocar de provedor não muda uma linha do seu bot.

Passo a passo no Desk: criar a chave

A configuração é feita por um administrador, em **Configurações › Integrações**.

1 Abra Configurações › Integrações

No topo está a **URL base** da API (use o botão de copiar). Abaixo, o formulário de nova chave e a lista de chaves ativas.

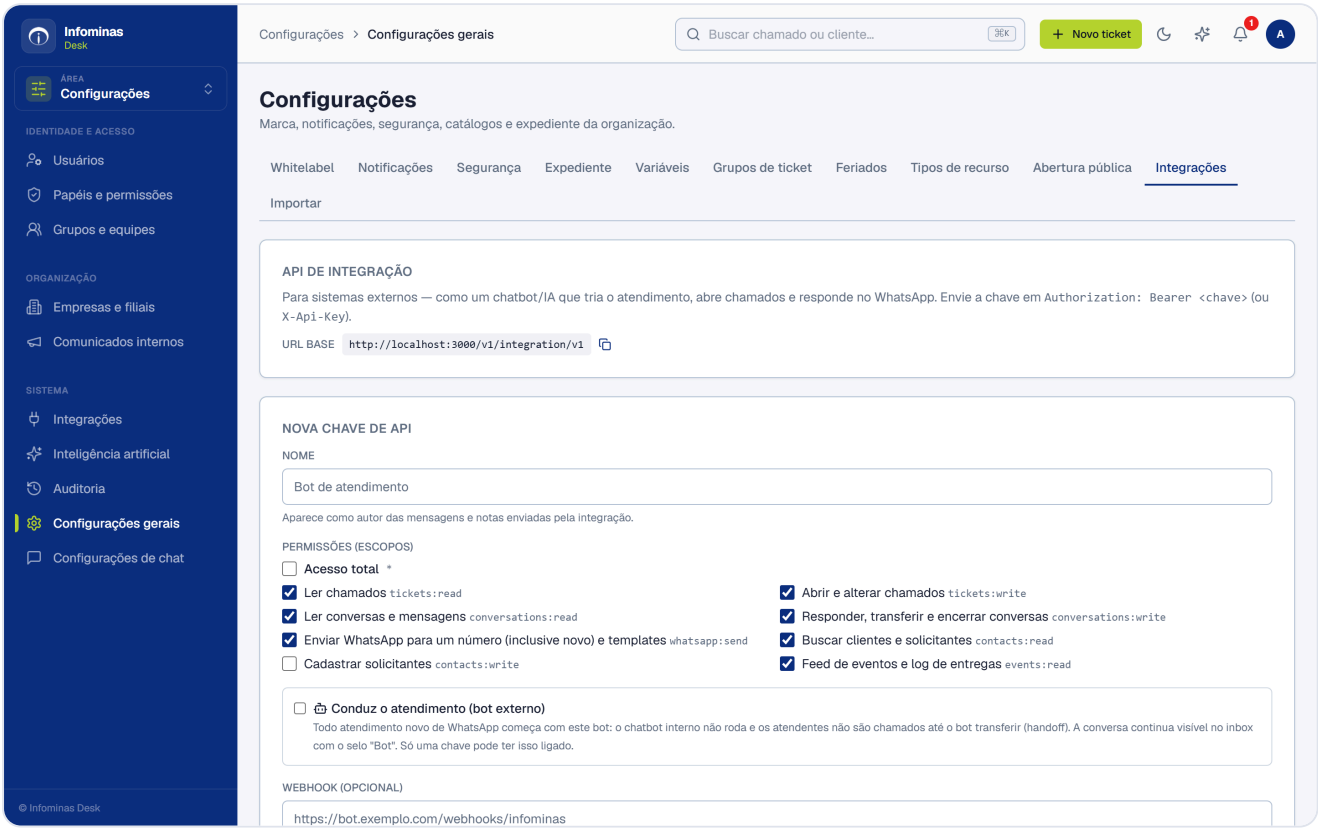


Figura 1. Configurações › Integrações: URL base, nova chave e chaves ativas.

2 Preencha a nova chave

Nome (aparece como autor das mensagens do bot), **permissões** (escolha o mínimo necessário ou *Acesso total*), **Conduz o atendimento** se esta integração é o bot do WhatsApp, e a **URL do webhook** com os eventos que você quer receber.

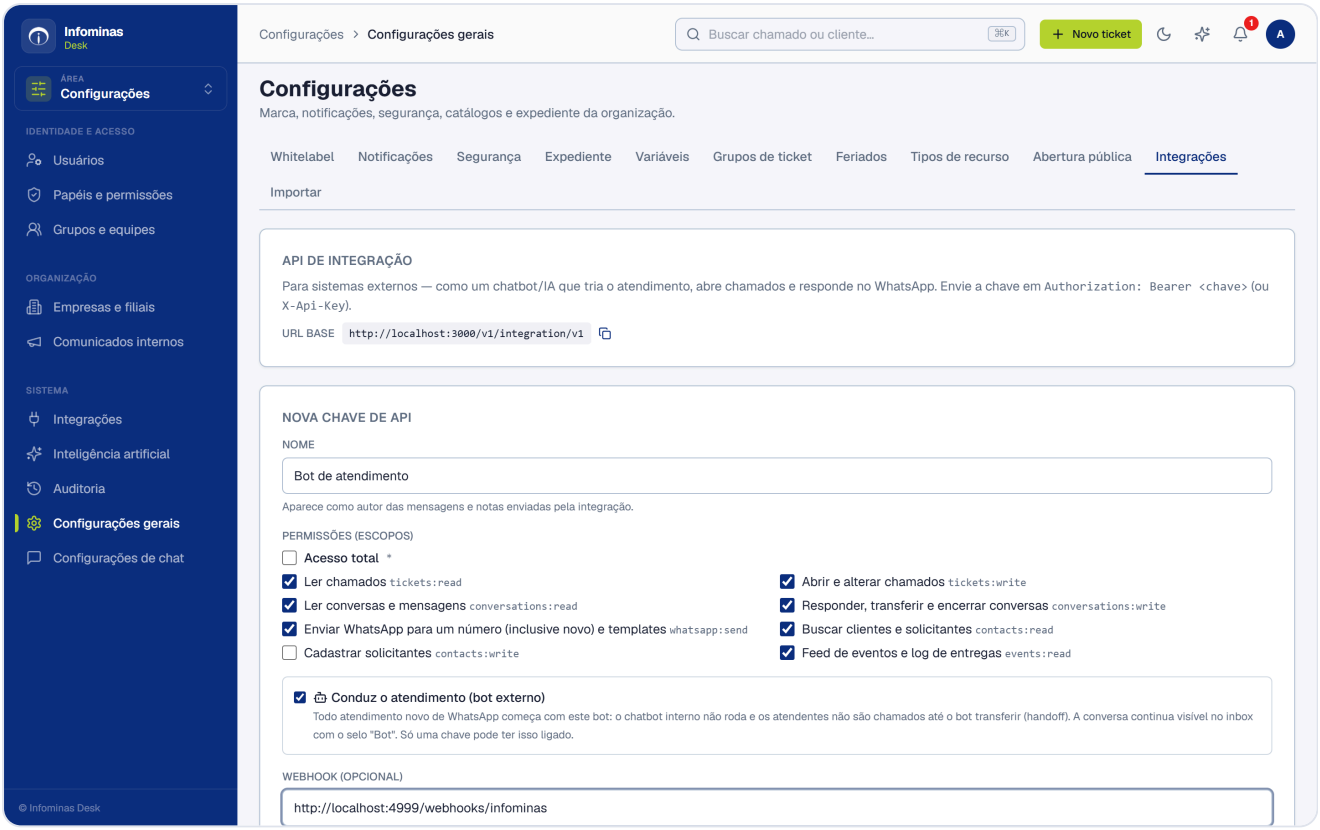


Figura 2. Formulário preenchido: escopos, modo bot externo, URL do webhook e eventos agrupados.

3 Copie a chave e o segredo

Ao criar, o Desk mostra a **chave de API** (`ik_live_…`) e o **segredo do webhook** (`whsec_…`) **uma única vez**. Guarde os dois no cofre de segredos do seu sistema.



Figura 3. Chave e segredo revelados uma única vez, com botão de copiar.

> A chave dá acesso aos dados da organização. Nunca a coloque em código de front-end, aplicativo móvel ou repositório. Vazou? **Revogue** na hora e crie outra.

4 Teste o webhook

Na chave, clique em **Enviar teste**: o Desk faz um `POST` assinado com o evento `ping` e mostra o código HTTP e o tempo de resposta.

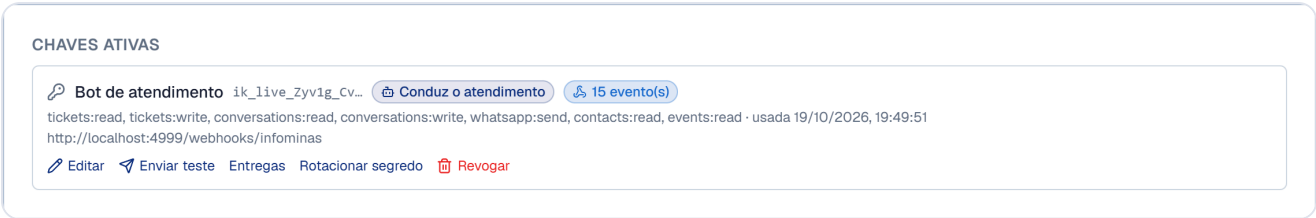


Figura 4. Chaves ativas com os selos de bot externo e webhook, e as ações Editar, Enviar teste, Entregas, Rotacionar segredo e Revogar.

5 Acompanhe as entregas

Em **Entregas** você vê cada evento enviado, o status (*Entregue*, *Na fila*, *Falhou*), as tentativas e a resposta do seu servidor, e pode **Reentregar**.

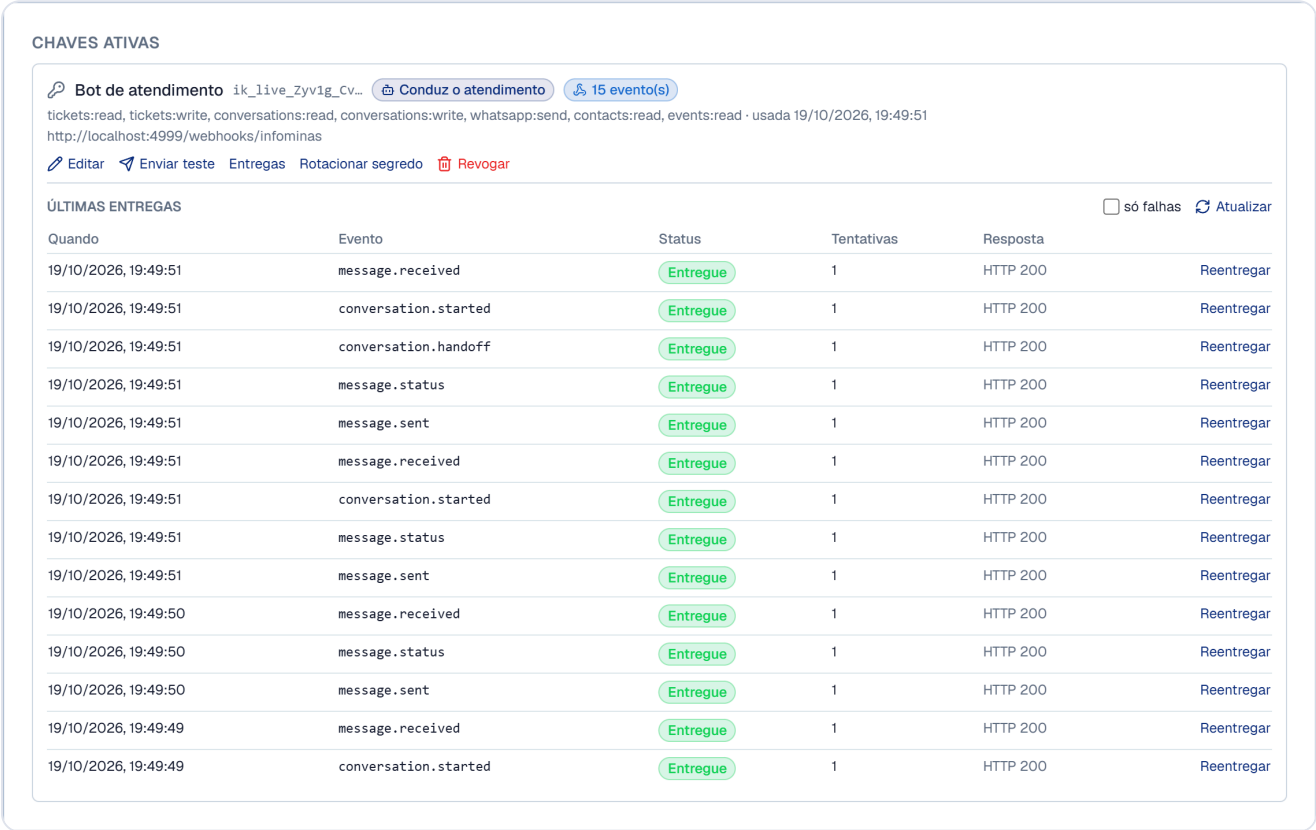


Figura 5. Log de entregas do webhook, com reentrega manual.

6 Edite quando precisar

Editar permite trocar escopos, eventos, URL e o modo bot sem gerar outra chave. Ao trocar de servidor, use **Rotacionar segredo**.

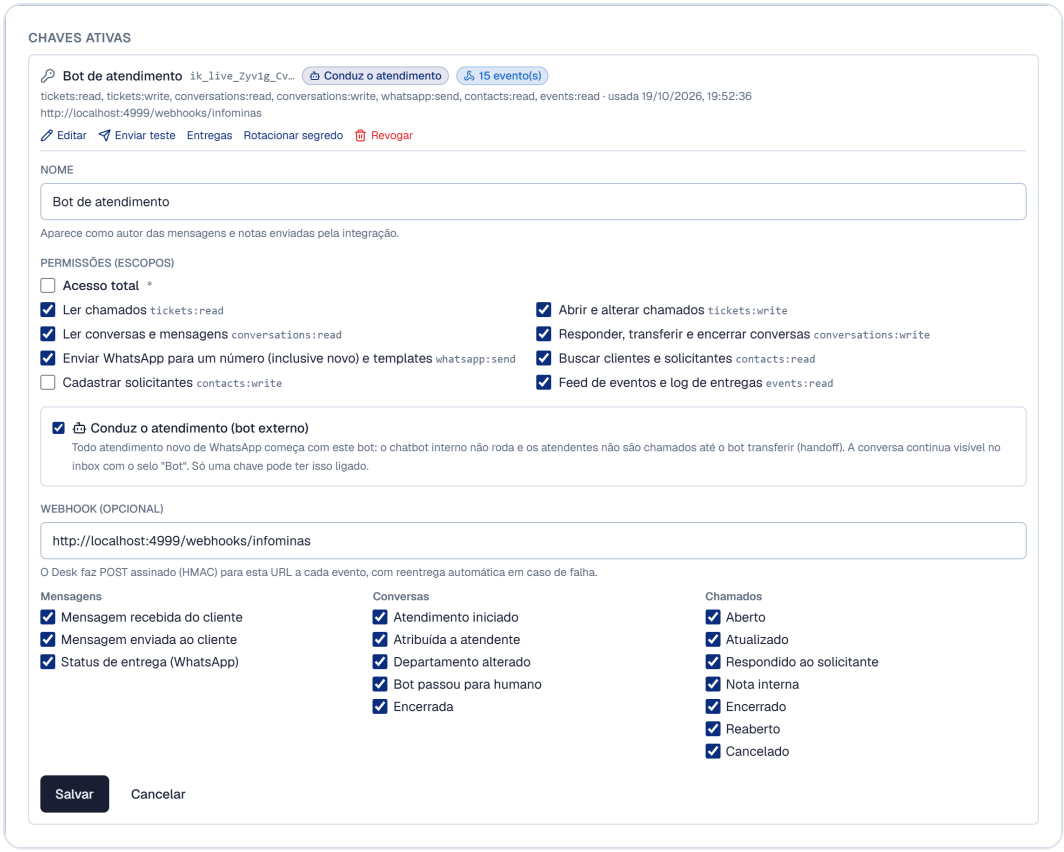


Figura 6. Edição da chave existente.

7 O que a equipe vê no inbox

Com o bot externo ligado, a conversa aparece no inbox com o selo **Bot**, sem som nem notificação, até o bot fazer o *handoff*. Um atendente pode assumir a qualquer momento.

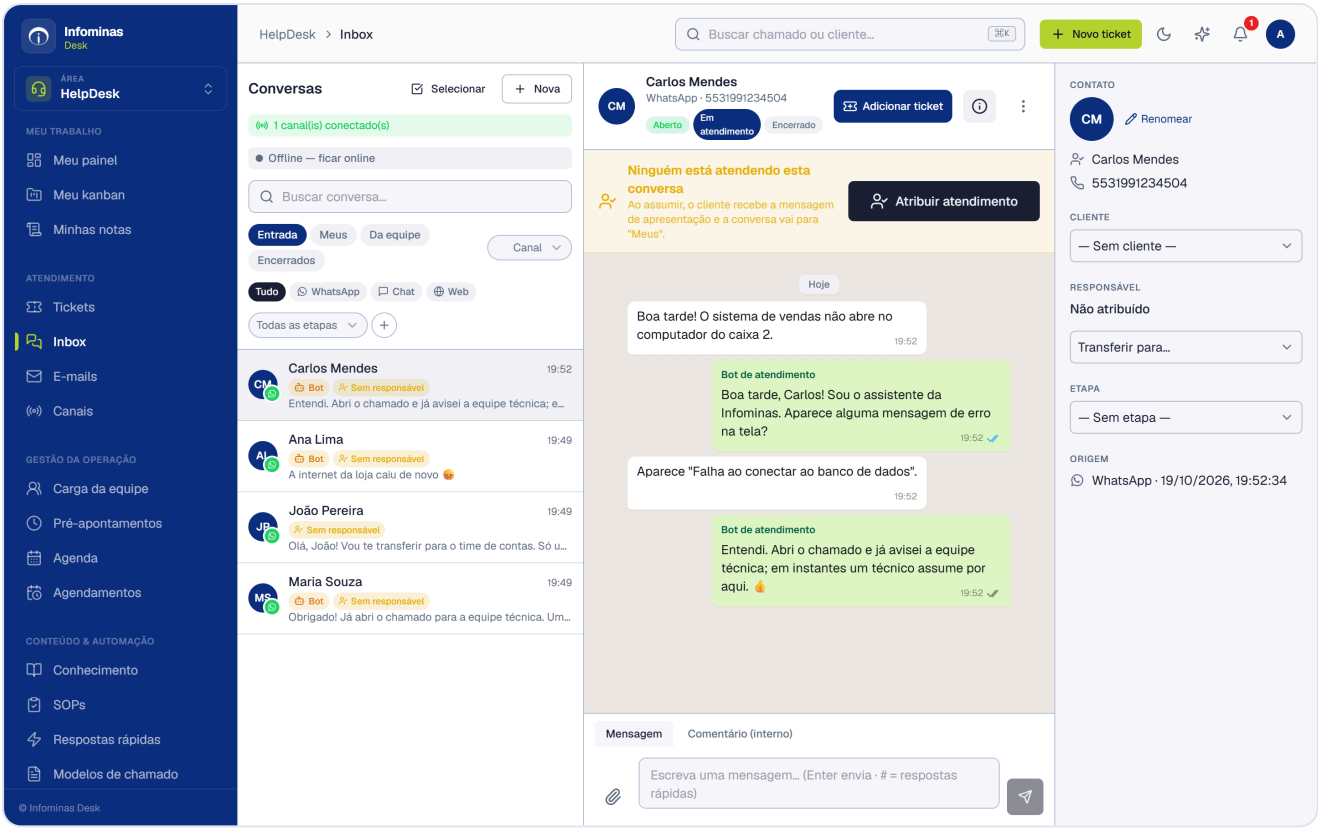


Figura 7. Inbox: conversa conduzida pelo bot externo, com as mensagens do bot assinadas pelo nome da chave.

8 Templates do WhatsApp

Para iniciar conversa ou falar fora da janela de 24h, importe os templates aprovados do provedor em **Templates do WhatsApp**. A API os chama pelo **nome**.

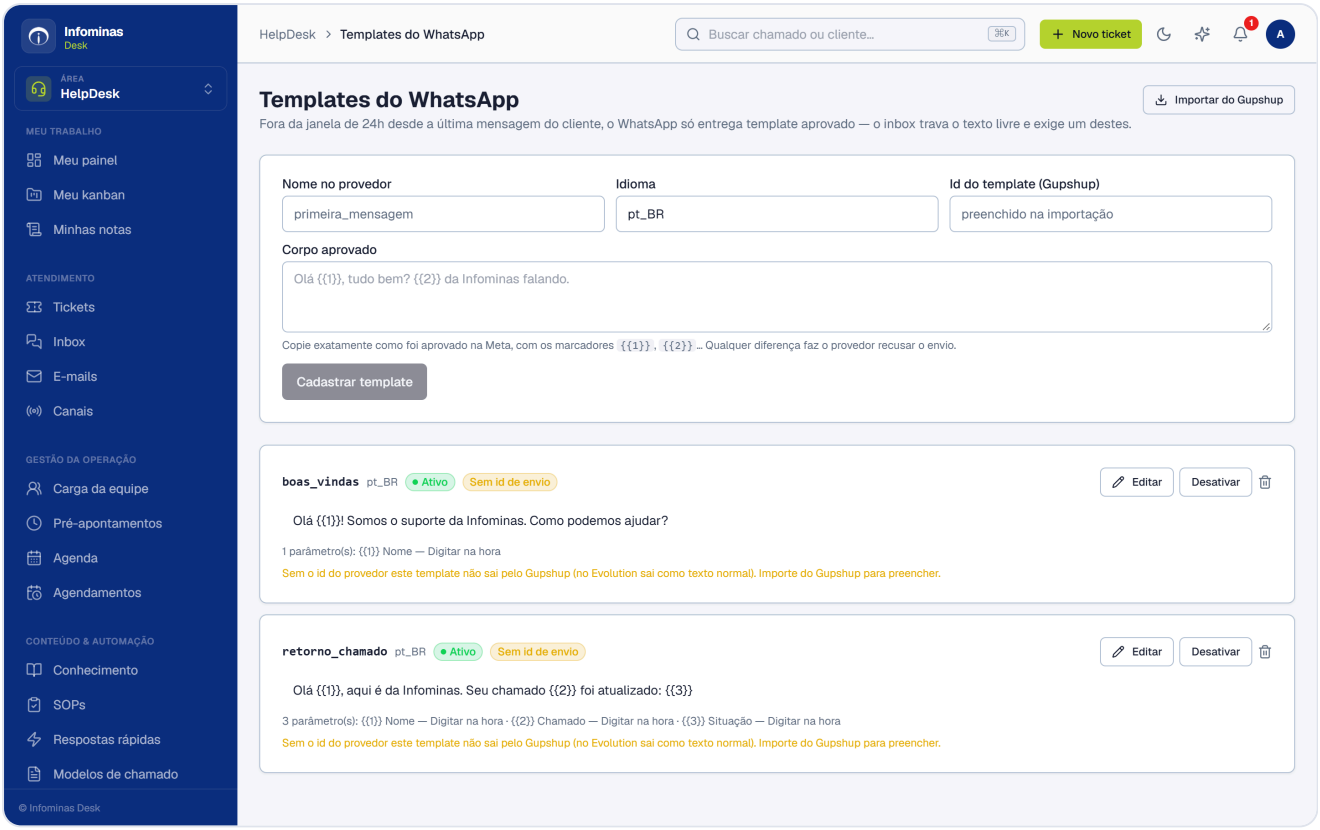


Figura 8. Templates do WhatsApp: importação dos templates aprovados no provedor.

2. Autenticação e escopos

Envie a chave em **todas** as chamadas, de uma das formas:

```http
Authorization: Bearer ik_live_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
X-Api-Key: ik_live_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Teste a chave com `GET /me`:

```json
{ "organizationId": "…", "keyId": "…", "name": "Bot de atendimento",
  "scopes": ["tickets:read", "conversations:write", "whatsapp:send"],
"handlesConversations": true }
```

Escopo	Libera
`tickets:read`	listar e consultar chamados
`tickets:write`	abrir, alterar, comentar, responder, encerrar, reabrir, cancelar, escalar chamados; alertas
`conversations:read`	listar e ler conversas e mensagens, retroativo de mensagens, anexos, janela de 24h

Escopo	Libera
`conversations:write`	enviar mensagem em conversa, nota interna, atribuir, transferir, encerrar, handoff, vínculos
`whatsapp:send`	enviar WhatsApp para um **número** (inclusive novo)
`contacts:read`	identificar telefone, buscar clientes e solicitantes
`contacts:write`	cadastrar solicitante
`events:read`	feed de eventos e log/reentrega de webhooks
`*`	tudo

Dados de referência (`/desks` , `/desks/:id` , `/users` , `/departments` , `/catalog` , `/whatsapp/templates`) valem para qualquer chave válida.

3. Erros

Todo erro tem o mesmo formato. Decida pelo **`code`** ; a `message` é para leitura humana.

```JSON
{ "statusCode": 409, "code": "WHATSAPP_WINDOW_CLOSED",
  "message": "A janela de 24h do WhatsApp está fechada: envie um template aprovado.",
  "lastInboundAt": "2026-10-18T09:12:00.000Z", "windowExpiresAt": "2026-10-19T09:12:00.000Z"
}
```

HTTP	**code**	Quando
400	`VALIDATION_ERROR`	corpo/parâmetro inválido (lista em `errors`)
400	`INVALID_REFERENCE`	id de outro registro inexistente
401	`UNAUTHORIZED`	chave ausente, inválida ou revogada
403	`FORBIDDEN`	escopo insuficiente
404	`NOT_FOUND`	recurso não existe nesta organização
404	`TEMPLATE_NOT_FOUND`	template inexistente ou desativado
409	`CONFLICT`	conflito (ex.: chamado preso a outro atendimento)

HTTP	code	Quando
409	`WHATSAPP_WINDOW_CLOSED`	texto/mídia fora da janela de 24h, ou número sem conversa: use template
409	`WHATSAPP_NOT_CONFIGURED`	nenhum canal de WhatsApp no Desk
422	`TEMPLATE_NOT_APPROVED`	template não aprovado/importado do provedor
422	`TEMPLATE_PARAMS_MISMATCH`	parâmetros faltando ou sobrando (`paramCount` , `paramLabels`)
400/422	`MEDIA_URL_REQUIRED` / `MEDIA_DOWNLOAD_FAILED`	URL de mídia inválida, interna, inacessível ou > 16 MB
429	`RATE_LIMITED`	limite de requisições (300/min por IP)
500	`INTERNAL_ERROR`	erro do Desk

Falha do **provedor** depois do envio (número inexistente, opt-out, template recusado pela Meta) chega como evento `message.status` com `status: "failed"` e `error.code: "PROVIDER_ERROR"` .

4. Paginação

Listas devolvem `{ "data": [...], "nextCursor": "…" }` . Para a próxima página, repita a chamada com `cursor=<nextCursor>` ; `nextCursor: null` = fim. `limit` padrão 50 (máx. 100, salvo indicação).

5. Eventos e webhooks

5.1 Como funciona

Cada acontecimento vira um **evento** gravado no Desk (na mesma transação do fato, então não se perde) com um número sequencial `seq`. Se a chave tem webhook, o Desk faz **POST** do evento para a URL:

- **Sucesso** = qualquer 2xx em até **8 s**. Responda rápido e processe de forma assíncrona.
- **Reentrega automática** em caso de falha: após 30 s, 2 min, 10 min, 1 h e 6 h. Depois disso a entrega fica como **Falhou** (reentregável na tela ou por `POST /deliveries/:id/redeliver`).
- **Pelo menos uma vez:** o mesmo evento pode chegar mais de uma vez, então **deduplique pelo `id`**. A ordem também não é garantida entre reentregas: ordene por `seq` se precisar.
- **Retenção:** 30 dias.

5.2 Eventos

Evento	Quando
`message.received`	mensagem do cliente: WhatsApp (individual e grupo), chat do agente desktop, portal
`message.sent`	mensagem ao cliente: atendente, bot interno, esta API, eco do app do WhatsApp, resposta do chamado
`message.status`	entrega no WhatsApp: `sent` → `delivered` → `read`, ou `failed`
`conversation.started`	atendimento novo (1ª mensagem ou nova sessão após 24 h/encerramento)
`conversation.assigned`	conversa atribuída/transferida a um atendente
`conversation.department_changed`	departamento alterado (`previousDepartmentId`)
`conversation.handoff`	o bot externo passou a conversa para humanos (`note`)
`conversation.finished`	conversa encerrada (`reason` : `inactivity` , `bot` , ou ausente = atendente/API)
`ticket.created`	chamado aberto (qualquer origem)
`ticket.updated`	campos alterados (`changes` : lista de campos) ou transição (`change`)
`ticket.replied`	resposta pública ao solicitante (`reply.body`)
`ticket.note_added`	nota interna (`note.body`)

Evento	Quando
`ticket.closed` / `ticket.reopened` / `ticket.canceled`	transições
`ping`	teste ("Enviar teste" / `POST /webhook/test`)

5.3 Envelope

```json
{
  "id": "evt_000000000123",
  "seq": 123,
  "event": "message.received",
  "createdAt": "2026-10-19T12:00:00.000Z",
  "organizationId": "8f0c…",
  "data": { … }
}
```

Eventos de mensagem (`data`):

```json
{
  "conversation": {
    "id": "c1d2…", "channel": "WHATSAPP", "status": "PENDING", "subject": null,
    "botActive": true, "handledBy": "EXTERNAL_BOT",
    "assignedUserId": null, "departmentId": null, "clientId": null, "requesterId": null,
    "resourceId": null, "ticketNumber": null,
    "contact": { "name": "Maria", "phone": "+5531990000000", "email": null, "groupJid": null
},
    "provider": "GUPSHUP", "lastMessageAt": "…", "lastMessagePreview": "Bom dia",
    "unreadCount": 1, "sessionStartedAt": "…", "createdAt": "…", "updatedAt": "…"
  },
  "contact": { "name": "Maria", "phone": "+5531990000000", "email": null, "groupJid": null
},
  "message": {
    "id": "m9…", "conversationId": "c1d2…", "sender": "CLIENT", "senderName": null,
    "type": "TEXT", "status": null, "body": "Bom dia, minha impressora parou",
    "internal": false, "deleted": false, "replyToId": null, "createdAt": "…",
    "attachment": null
  }
}
```

- `handledBy`: `EXTERNAL_BOT` (bot externo conduz), `INTERNAL_BOT` (chatbot do Desk), `AGENT` (atendente), `QUEUE` (na fila).
- `sender`: `CLIENT`, `AGENT` (atendente, bot ou integração) ou `SYSTEM`.
- `type`: `TEXT`, `IMAGE`, `FILE`, `AUDIO` ou `CONTACT`. Mídia recebida chega como mensagem com `attachment.url`; baixe com a chave (`GET /attachments/:id`). No WhatsApp, a mídia gera **duas** mensagens: o texto/legenda (ou um rótulo como "[imagem]") e o arquivo.

- `message.status` traz também `"status": "sent|delivered|read|failed"` e `"error": { "code", "detail" } | null`.

Eventos de conversa: `{ conversation, contact, …extras }`. **Eventos de chamado:** `{ ticket, … extras }` (o objeto `ticket` está na seção 9).

5.4 Verificar a assinatura

Cada entrega traz os headers:

```http
X-Infominas-Event: message.received
X-Infominas-Delivery: 5b0e…           (id da entrega)
X-Infominas-Timestamp: 1760875200     (segundos Unix)
X-Infominas-Signature: sha256=<hex>
```

A assinatura é `HMAC-SHA256(webhookSecret, "<timestamp>.<corpo bruto>")`. Valide sobre o **corpo bruto**, antes de fazer parse, e recuse timestamps com mais de 5 minutos (proteção contra replay).

Node.js (Express):

```js
import crypto from "node:crypto";
import express from "express";

const app = express();
app.post("/webhooks/infominas", express.raw({ type: "application/json" }), (req, res) => {
  const ts = req.header("X-Infominas-Timestamp");
  const sig = req.header("X-Infominas-Signature") ?? "";
  const expected = "sha256=" + crypto.createHmac("sha256",
process.env.INFOMINAS_WEBHOOK_SECRET)
    .update(`${ts}.${req.body}`).digest("hex");
  const fresh = Math.abs(Date.now() / 1000 - Number(ts)) < 300;
  if (!fresh || sig.length !== expected.length || !crypto.timingSafeEqual(Buffer.from(sig),
Buffer.from(expected))) {
    return res.sendStatus(401);
  }
  res.sendStatus(200);                       // responda já…
  queue.push(JSON.parse(req.body));          // …e processe depois (deduplique por event.id)
});
```

Python (FastAPI):

```python
import hmac, hashlib, time, os
from fastapi import FastAPI, Request, Response

app = FastAPI()
SECRET = os.environ["INFOMINAS_WEBHOOK_SECRET"].encode()

@app.post("/webhooks/infominas")
async def infominas(request: Request):
    raw = await request.body()
    ts = request.headers.get("X-Infominas-Timestamp", "0")
    sig = request.headers.get("X-Infominas-Signature", "")
    expected = "sha256=" + hmac.new(SECRET, f"{ts}.".encode() + raw,
hashlib.sha256).hexdigest()
    if abs(time.time() - int(ts)) > 300 or not hmac.compare_digest(sig, expected):
        return Response(status_code=401)
    enqueue(raw)            # processe de forma assíncrona; deduplique por id
    return Response(status_code=200)
```

5.5 Retroativo: nunca perder nada

Rota	Escopo	Uso
`GET /events?after=<seq>&limit=&types=&conversationId=&ticketNumber=`	`events:read`	feed de eventos em ordem. Guarde o maior `seq` processado e peça `after=<seq>`
`GET /messages?since=<ISO>&channel=&limit=&cursor=`	`conversations:read`	todas as mensagens desde um instante (máx. 500/página)
`GET /conversations/:id/messages?after=<messageId>&limit=`	`conversations:read`	mensagens de uma conversa em ordem cronológica
`GET /deliveries?status=FAILED`	`events:read`	entregas de webhook desta chave
`POST /deliveries/:id/redeliver`	`events:read`	recoloca a entrega na fila
`POST /webhook/test`	—	envia um `ping` assinado e devolve `{ ok, statusCode, error, durationMs }`

Exemplo de reconciliação (a cada minuto, ou ao subir o bot):

```bash
curl -s "$BASE/events?after=$ULTIMO_SEQ&limit=500" -H "Authorization: Bearer $KEY"
# → { "data": [ {envelope}, … ], "nextAfter": 1290, "hasMore": false }
```

O webhook e o feed entregam **o mesmo envelope**, então o mesmo handler serve para os dois.

6. WhatsApp e conversas

6.1 Janela de 24h e templates

O WhatsApp só aceita **texto livre ou mídia** até 24h depois da **última mensagem do cliente**. Fora dessa janela, e para iniciar conversa com um número que nunca falou, só vale **template aprovado**. O Desk aplica a regra igual para qualquer provedor e responde `409 WHATSAPP_WINDOW_CLOSED`.

- `GET /whatsapp/templates`: templates habilitados.

```json
[{ "id": "…", "name": "retorno_chamado", "language": "pt_BR",
   "body": "Olá {{1}}, seu chamado {{2}} foi atualizado.", "paramCount": 2,
   "paramLabels": ["Nome", "Chamado"], "approved": true }]
```

- `GET /whatsapp/window?phone=+5531990000000` (`conversations:read`):

```json
{ "phone": "+55319900000000", "open": false, "expiresAt": null, "lastInboundAt": null,
"conversationId": null }
```

6.2 Enviar para um número: `POST /whatsapp/messages` (`whatsapp:send`)

Abre ou reaproveita a conversa de WhatsApp do número e envia. Use um destes conteúdos:

```json
{ "to": "+55319900000000", "contactName": "Maria", "text": "Olá! Posso ajudar?" }
```

```json
{ "to": "+55319900000000", "template": { "name": "retorno_chamado", "language": "pt_BR",
"params": ["Maria", "#4521"] } }
```

```json
{ "to": "+5531990000000", "media": { "url": "https://arquivos.exemplo.com/boleto.pdf",
"filename": "boleto.pdf", "caption": "Segue o boleto" } }
```

Campos opcionais: `clientId`, `requesterId` (vínculos da conversa) e `bot` (`true` = a conversa fica com o bot externo; o padrão é o modo da chave). Resposta `201`:

```json
{ "conversationId": "c1d2…",
  "messages": [{ "id": "m9…", "sender": "AGENT", "senderName": "Bot de atendimento", "type":
"TEXT",
                "body": "Olá! Posso ajudar?", "status": "queued", "…": "…" }] }
```

O resultado do provedor chega depois pelo evento `message.status`.

6.3 Responder numa conversa: `POST /conversations/:id/messages` (`conversations:write`)

Mesmo corpo (sem `to`): `text`, `template` (só WhatsApp) ou `media` (+ `caption`), e opcionalmente `replyToId` (responder citando). Com `"internal": true` + `text` grava uma **nota interna** (só a equipe vê). Funciona em qualquer canal: WhatsApp, chat do agente desktop, portal. As regras de janela valem só para WhatsApp individual.

Mensagem enviada pela integração **não tira o bot de cena** e leva o nome da chave como autor.

6.4 Ler e operar conversas

Rota	Escopo	
`GET /conversations`	`conversations:read`	filtros: `tab` (`entrada` = sem dono, `abertas`, `encerrados`, `todas`), `status` (`PENDING` / `IN_PROGRESS` / `FINISHED`), `channel` (`WHATSAPP` / `CHAT` / `WEB` / `EMAIL`), `assignedUserId` (ou `none`), `departmentId`, `botActive`, `phone`, `updatedSince`, `limit`, `cursor`
`GET /conversations/:id`	`conversations:read`	conversa + `whatsappWindow` + `recentMessages` (últimas 50, sem notas internas)
`POST /conversations/:id/assign { "userId" }`	`conversations:write`	atribui a um atendente (avisa o cliente conforme a config de chat)
`POST /conversations/:id/department { "departmentId" }`	`conversations:write`	transfere de departamento (`null` remove)
`POST /conversations/:id/finish`	`conversations:write`	encerra (sem mensagem ao cliente)
`POST /conversations/:id/client { "id" }`	`conversations:write`	vincula cliente (`null` desfaz)
`POST /conversations/:id/requester { "id" }`	`conversations:write`	vincula solicitante (traz o cliente junto)
`GET /attachments/:id`	`conversations:read`	baixa o arquivo de uma mensagem

7. Bot externo (triagem)

Com **Conduz o atendimento (bot externo)** ligado na chave (só uma chave por organização):

1. Todo atendimento **novo** de WhatsApp individual (1ª mensagem do número, ou nova sessão após 24 h ou após o encerramento) **nasce com o bot**: `handledBy: "EXTERNAL_BOT"` já no `message.received`.
2. O **chatbot interno não roda** e **os atendentes não são chamados** (sem som, push nem sino). A conversa aparece no inbox com o selo **Bot**, e um atendente pode assumir a qualquer momento (isso tira o bot).
3. O bot conversa por `POST /conversations/:id/messages`.
4. Para passar a humanos, use **handoff**:

   ```HTTP
   POST /conversations/:id/handoff
   { "departmentId": "…", "note": "Cliente Maria (ACME). Impressora HP do financeiro sem imprimir desde ontem. Chamado #4521 aberto." }
   ```

   O bot sai, a conversa volta para a fila (ou vai para o `userId` informado, ou para a distribuição automática da organização), a `note` vira nota interna e os atendentes são notificados. Evento `conversation.handoff`.
5. `POST /conversations/:id/bot { "active": true|false }` assume ou solta a conversa sem avisar ninguém.

Grupos de WhatsApp e o chat do agente desktop (que é sobre chamado já aberto) não passam pelo bot.

8. Contatos e solicitantes

Rota	Escopo	
`GET /contacts/lookup?phone=+5531990000000`	`contacts:read`	solicitantes, contatos de cliente, `clientIds`, chamados **abertos** e a conversa ativa do número
`GET /clients?q=ACME` ou `?document=12.345.678/0001-90`	`contacts:read`	clientes por nome/razão social ou CPF/CNPJ
`GET /requesters?phone=&email=&q=&clientId=`	`contacts:read`	busca solicitantes
`GET /requesters/:id`	`contacts:read`	solicitante + cliente + `openTickets`
`POST /requesters`	`contacts:write`	cadastra **sem duplicar** e, opcionalmente, já vincula (abaixo)

Rota	Escopo	
`PATCH /requesters/:id { "name?", "email?", "phone?", "extension?", "clientId?" }`	`contacts:write`	corrige o cadastro (`""` apaga e-mail/telefone/ramal)
`POST /conversations/:id/requester { "id" }`	`conversations:write`	vincula à conversa (`null` desfaz); o cliente vem junto
`POST /tickets/:number/requester { "id" }`	`tickets:write`	vincula ao chamado (`null` desfaz); o cliente vem junto se o chamado não tiver

8.1 Cadastrar e vincular o solicitante numa chamada só

```http
POST /requesters
{ "name": "Maria Souza", "clientId": "k1…", "phone": "+5531990000000", "email":
"maria@acme.com.br",
  "conversationId": "c1d2…", "ticketNumber": 4521 }
```

Resposta `200` (o solicitante, com `created`):

```json
{ "id": "r1…", "name": "Maria Souza", "email": "maria@acme.com.br", "phone":
"+5531990000000",
  "extension": null, "clientId": "k1…", "canOpenWhatsapp": true,
  "client": { "id": "k1…", "name": "ACME Ltda" }, "openTickets": 1, "created": true, "…":
"…" }
```

- **Idempotente:** se o cliente já tem alguém com o mesmo telefone (qualquer forma: com/sem `+55`, com/sem o 9º dígito) ou o mesmo e-mail, a API devolve esse cadastro com `created: false` e não cria outro. Pode repetir a chamada sem medo (retry, webhook reentregue).
- `conversationId` e `ticketNumber` são opcionais: com eles, o solicitante já sai vinculado à conversa e/ou ao chamado (e o cliente dele é preenchido onde faltar).
- Fluxo típico do bot: `GET /contacts/lookup?phone=` → não achou → pergunta nome e empresa → `GET /clients?q=` → `POST /requesters` com `conversationId`.

8.2 Identificar um telefone

JSON

```
{
  "phone": "+5531990000000",
  "requesters": [{ "id": "r1…", "name": "Maria Souza", "email": "maria@acme.com.br",
"phone": "+5531990000000",
                  "clientId": "k1…", "client": { "id": "k1…", "name": "ACME Ltda" } }],
  "clientContacts": [],
  "clientIds": ["k1…"],
  "openTickets": [ { "number": 4510, "title": "VPN não conecta", "…": "…" } ],
  "conversation": { "id": "c1d2…", "handledBy": "EXTERNAL_BOT", "…": "…" }
}
```

9. Chamados

9.1 Objeto `ticket`

```json
{
  "id": "t1…", "number": 4521, "title": "Impressora do financeiro não imprime",
  "description": "…", "channel": "WHATSAPP", "ticketType": "INCIDENT",
  "impact": null, "urgency": null, "priorityLevel": null,
  "desk": { "id": "d1…", "name": "Suporte" },
  "status": { "id": "s1…", "name": "Aberto", "isOpen": true, "isClose": false, "isCanceled": false },
  "stage": { "id": "e1…", "name": "Triagem" },
  "priority": { "id": "p1…", "name": "Alta" },
  "catalogItemId": null,
  "client": { "id": "k1…", "name": "ACME Ltda" },
  "requester": { "id": "r1…", "name": "Maria Souza", "email": "…", "phone": "+5531990000000" },
  "resourceId": null, "responsible": { "id": "u1…", "username": "joao", "email": "…" },
  "contactPhone": null, "conversationId": "c1d2…",
  "isClosed": false, "closedAt": null, "isDraft": false,
  "serviceDueAt": "…", "solutionDueAt": "…", "firstRespondedAt": null,
  "customValues": null, "createdAt": "…", "updatedAt": "…"
}
```

`GET /tickets/:number` traz também `messages` (histórico público do chamado).

9.2 Rotas

Rota	Escopo	
`GET /tickets?status=open|closed|all&deskId&clientId&requesterId&responsibleId&phone&updatedSince&limit&cursor`	`tickets:read`	lista (padrão: abertos)
`GET /tickets/:number`	`tickets:read`	chamado + histórico público
`POST /tickets`	`tickets:write`	abre (campos abaixo)
`PATCH /tickets/:number`	`tickets:write`	`title`, `description`, `statusId`, `stageId`, `priorityId`, `responsibleId`, `catalogItemId`, `clientId`, `requesterId`, `impact`, `urgency` (com `reason` ao mudar a prioridade ITIL), `customValues`
`POST /tickets/:number/notes` `{ "body" }`	`tickets:write`	nota interna

Rota	Escopo	
`POST /tickets/:number/reply { "body" }`	`tickets:write`	resposta ao solicitante (e-mail/WhatsApp, conforme o chamado)
`POST /tickets/:number/close`	`tickets:write`	encerra (respeita checklist/regras da mesa)
`POST /tickets/:number/reopen`	`tickets:write`	reabre
`POST /tickets/:number/cancel { "reason" }`	`tickets:write`	cancela
`POST /tickets/:number/escalate { "deskId" }`	`tickets:write`	muda de mesa
`POST /tickets/:number/requester { "id" }`	`tickets:write`	vincula/desvincula o solicitante (seção 8)
`POST /conversations/:id/ticket`	`tickets:write` + `conversations:write`	abre chamado a partir da conversa
`POST /conversations/:id/link-ticket { "ticketNumber", "finish?" }`	`conversations:write`	vincula a chamado existente (anexa a transcrição da sessão)

Abrir chamado (`POST /tickets`): `deskId` e `title` são obrigatórios. Opcionais: `description` , `channel` (`WHATSAPP` , `WEB` , `EMAIL` , `CHAT` , `AGENT`), `clientId` , `requesterId` , `contactPhone` , `priorityId` , `catalogItemId` , `statusId` , `stageId` , `responsibleId` , `ticketType` , `impact` , `urgency` , `customValues` . Se a organização exige campos (cliente, solicitante, serviço, prioridade), a falta vira `400` .

Chamado a partir da conversa (`POST /conversations/:id/ticket`):

```json
{ "deskId": "d1…", "title": "Impressora do financeiro não imprime",
  "description": "Resumo da triagem: HP M428 do financeiro, erro 49. Reiniciar não resolveu.",
  "priorityId": "p1…", "catalogItemId": "i1…", "period": "session", "finish": false }
```

A transcrição da conversa (`period` : `session` padrão, `today` , `24h` , `all`) entra na descrição, abaixo do resumo. Com `finish: true` a conversa é encerrada e o cliente recebe a mensagem configurada com o número do chamado.

9.3 Referência

Rota	
`GET /desks`	mesas ativas

Rota	
`GET /desks/:id`	mesa com `statuses`, `stages`, `priorities` e `catalogItems` (se a mesa usa catálogo)
`GET /catalog?deskId=`	serviços do catálogo
`GET /users`	atendentes ativos (`id`, `username`, `email`, `type`, `departmentIds`)
`GET /departments`	departamentos de chat

10. Fluxo de referência: bot de triagem

```
Cliente manda "oi" no WhatsApp
  └→ webhook message.received (handledBy=EXTERNAL_BOT, conversation.started antes)
       1. GET /contacts/lookup?phone=…          → quem é, cliente, chamados abertos
       2. POST /conversations/:id/messages      → "Olá Maria! É sobre o chamado #4510 ou
algo novo?"
       3. (cliente responde → message.received)
       4a. Assunto novo:
             POST /conversations/:id/requester {id}      (se identificado)
             ou POST /requesters {name, clientId, phone, conversationId}  (se novo)
             POST /conversations/:id/ticket {deskId, title, description, priorityId}
             POST /conversations/:id/messages "Abri o chamado #4521 para você."
       4b. Chamado existente:
             POST /conversations/:id/link-ticket {ticketNumber: 4510}
             POST /tickets/4510/notes "Cliente informou que…"
       5. Precisa de humano?
             POST /conversations/:id/handoff {departmentId, note: "resumo"}
          Resolveu sozinho?
             POST /conversations/:id/finish
  └→ reconciliação periódica: GET /events?after=<último seq>
```

Proativo (o Desk inicia):

```
POST /whatsapp/messages { to, template: { name: "retorno_chamado", params: [...] } }
  └→ message.status sent → delivered → read
  └→ cliente responde → message.received (janela aberta: texto livre liberado)
```

11. Outras rotas

- `POST /alerts` (`tickets:write`): alerta de ferramenta externa (RMM/firewall/backup) vira chamado, deduplicado por origem + tipo + recurso.

12. Boas práticas

- Responda o webhook em menos de 8 s e processe numa fila.
- Deduplique por `id` do evento; reconcilie por `GET /events?after=` ao iniciar e periodicamente.
- Guarde o `conversationId` do atendimento; use `handledBy` / `botActive` para saber se ainda é sua vez de falar.
- Antes de mandar texto livre para quem não falou nas últimas 24 h, consulte `/whatsapp/window` ou trate `409 WHATSAPP_WINDOW_CLOSED` enviando o template.
- Uma chave por sistema, com o mínimo de escopos. Revogue chaves vazadas na hora; rotacione o segredo do webhook quando trocar de servidor.